

MI GFX API

Version 3.2

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	11/30/2020
	Added PROCFS introduction	08/25/2021
3.1	Added more YUV422 format	03/20/2022
3.2	Added bitblt mode	05/06/2022

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. Overview1

- 1.1. Module Description 1
- 1.2. Flow diagram of invocation4
- 1.3. Keyword Description 5

2. API LIST 1

- 2.1. MI_GFX_Open 2
- 2.2. MI_GFX_Close 2
- 2.3. MI_GFX_WaitAllDone 3
- 2.4. MI_GFX_QuickFill 4
- 2.5. MI_GFX_GetARGB8888To1555AlphaThreshold 5
- 2.6. MI_GFX_SetARGB8888To1555AlphaThreshold 6
- 2.7. MI_GFX_BitBlit 7
- 2.8. MI_GFX_SetPalette 9
- 2.9. MI_GFX_CreateDev 10
- 2.10. MI_GFX_DestroyDev 11
- 2.11. MI_GFX_GetARGB1555To8888AlphaValue 12
- 2.12. MI_GFX_SetARGB1555To8888AlphaValue 13
- 2.13. MI_GFX_GetInitialScalingCoeff 14
- 2.14. MI_GFX_SetInitialScalingCoeff 14
- 2.15. MI_GFX_DrawLine 15

3. GFX DATA TYPE 18

- 3.1. MI_GFX_ColorFmt_e 19
- 3.2. MI_GFX_Surface_t 19
- 3.3. MI_GFX_Rect_t 20
- 3.4. MI_GFX_ColorKeyOp_e 21
- 3.5. MI_GFX_ColorKeyValue_t 22
- 3.6. MI_GFX_ColorKeyInfo_t 23
- 3.7. MI_GFX_Rotate_e 23
- 3.8. MI_GFX_Mirror_e 24
- 3.9. MI_GFX_DfbBldOp_e 24
- 3.10. MI_Gfx_DfbBlendFlags_e 26
- 3.11. MI_GFX_BitBlitMode_e 27
- 3.12. MI_GFX_Opt_t 28
- 3.13. MI_GFX_PaletteEntry_t 29
- 3.14. MI_GFX_Palette_t 29
- 3.15. MI_GFX_DevAttr_t 30
- 3.16. MI_GFX_Point_t30
- 3.17. MI_GFX_Line_t 31
- 3.18. MI_GFX_Dev 32

4. GFX ERROR CODE 33

5. PROCFS INTRODUCTION 34

5.1. cat 34

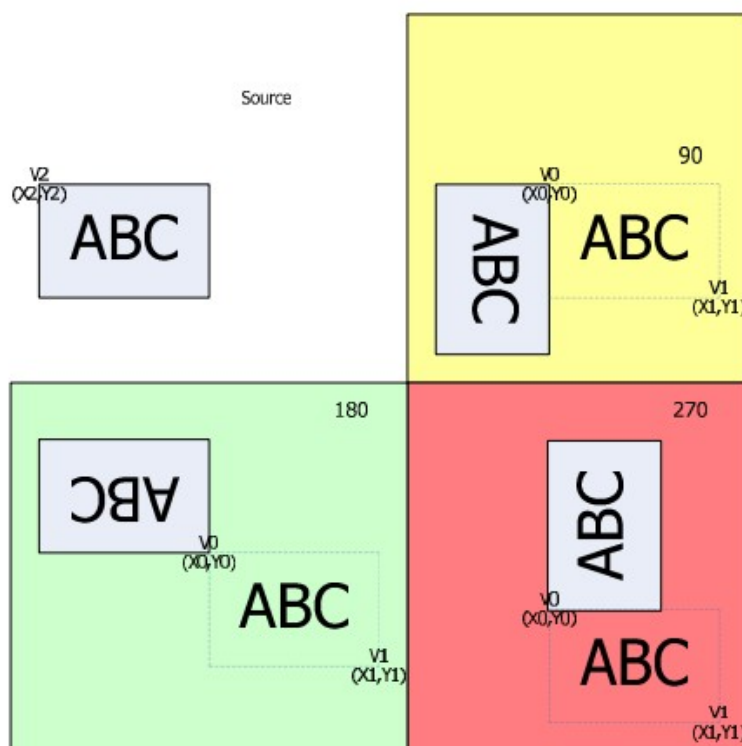
1. OVERVIEW

1.1. Module Description

GFX (Graphic Engine) hardware provides fast graphics rendering function to draw UI, mainly including rectangle color filling and bitmap moving which supports scaling, rotation, mirror flipping, format conversion, alpha blending and overlay, Color Key and etc.). It is supported by Taiyaki, Takoyaki, Tiramisu, Ikayaki, Muffin, Mochi.

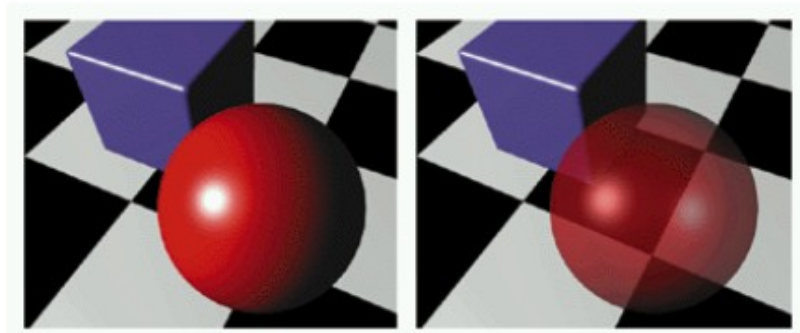
Rotate: The hardware supports 90/180/270 degree rotation.

Rotate

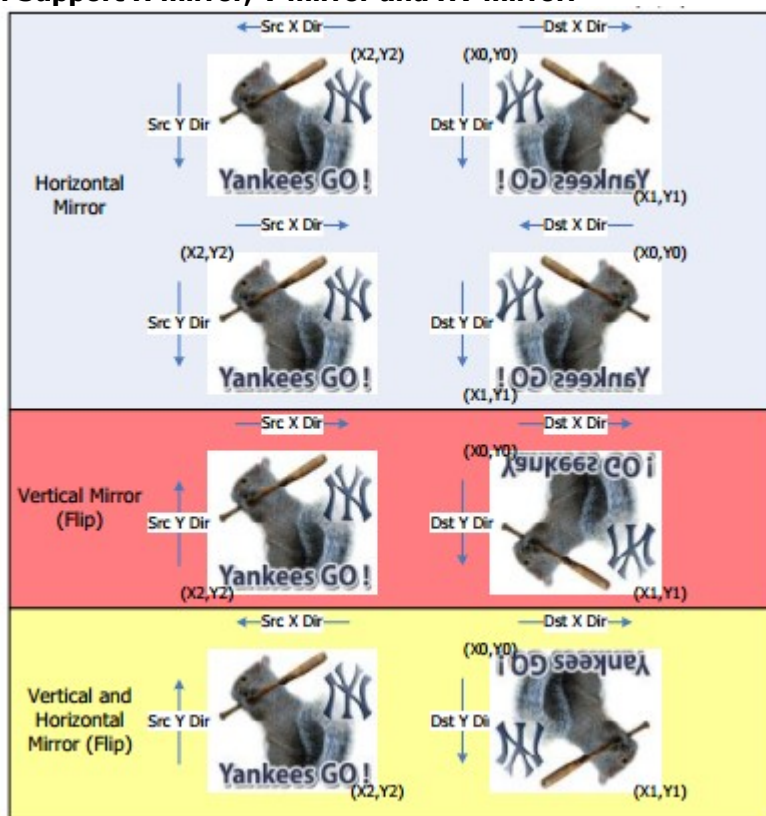


Alpha blending overlay: Supports normal 2D Alpha operation.

Alpha Blending

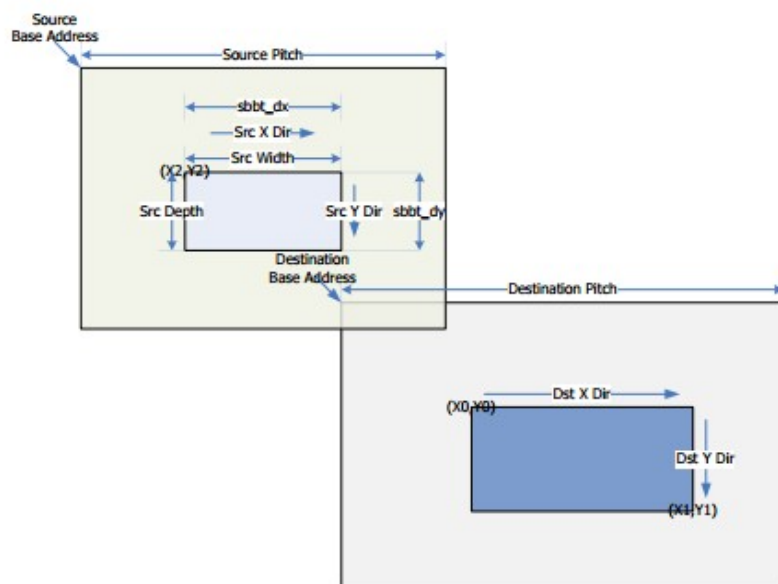


Mirror: Support H mirror, V mirror and HV mirror.



Zooming: Supports zooming at Bitblit.

Stretch Bitblt

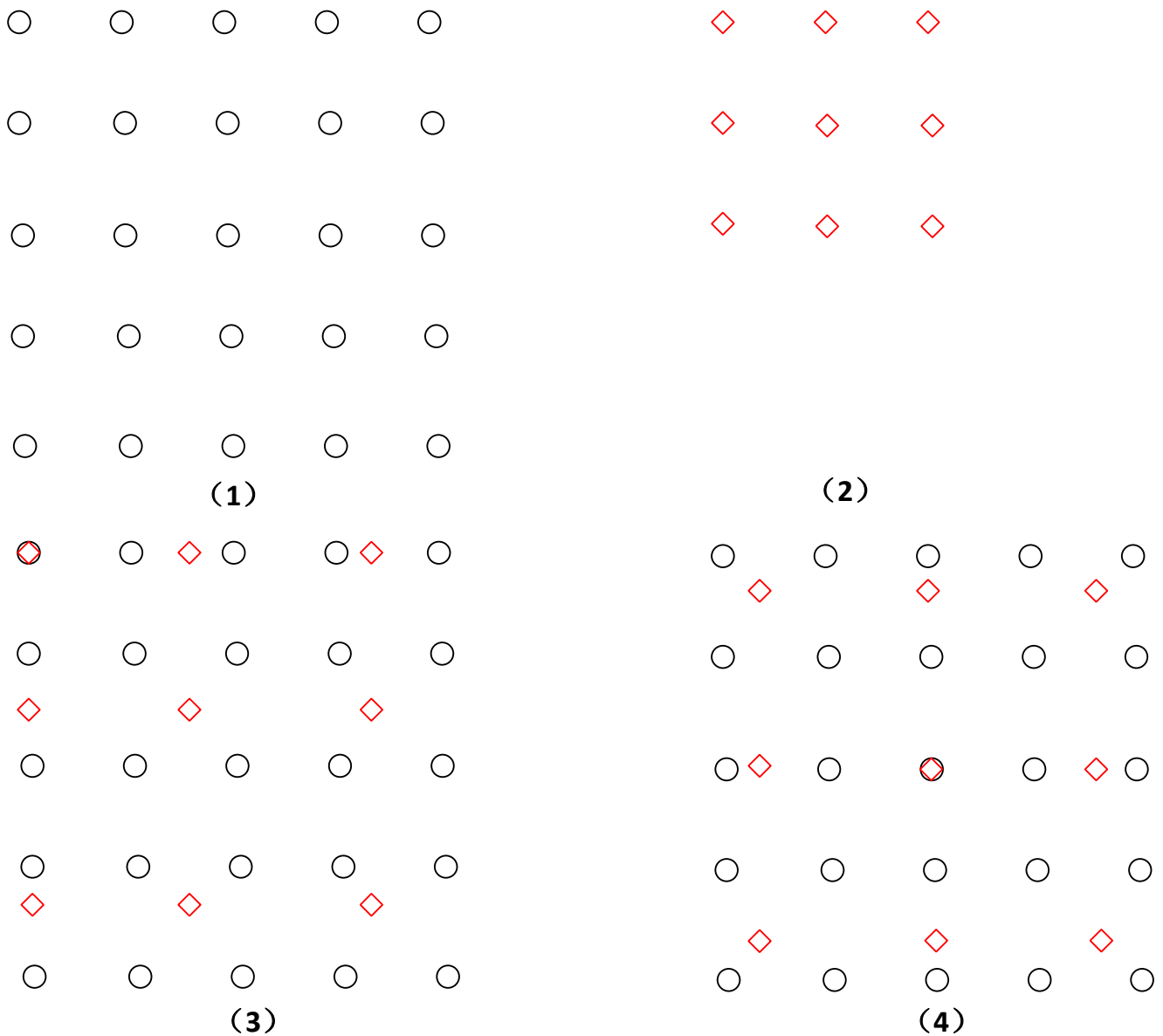


When using bilinear interpolation to scale, it involves the concept of geometric center shift of the image. The following figure is an example when a 5x5 image (figure (1)) needs to be shrunk to a 3x3 image (figure (2)). When the offset is not set, the pixel correspondence between the scaled image and the original image will be left overall as shown in Figure (3), while figure (4) shows the corresponding relation with the geometric center offset of the image.

Image geometric center:

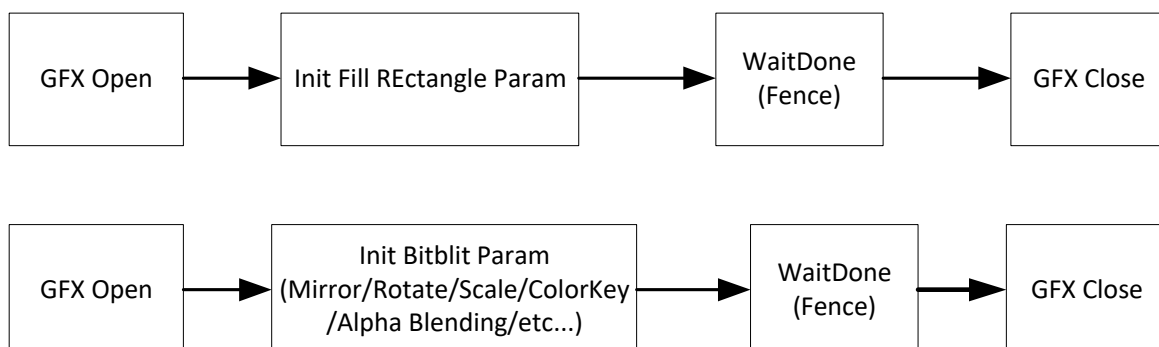
$$\text{SrcX} = (\text{dstX} + \text{coeff}) * (\text{srcWidth}/\text{dstWidth}) - \text{coeff}$$

$$\text{SrcY} = (\text{dstY} + \text{coeff}) * (\text{srcHeight}/\text{dstHeight}) - \text{coeff}$$

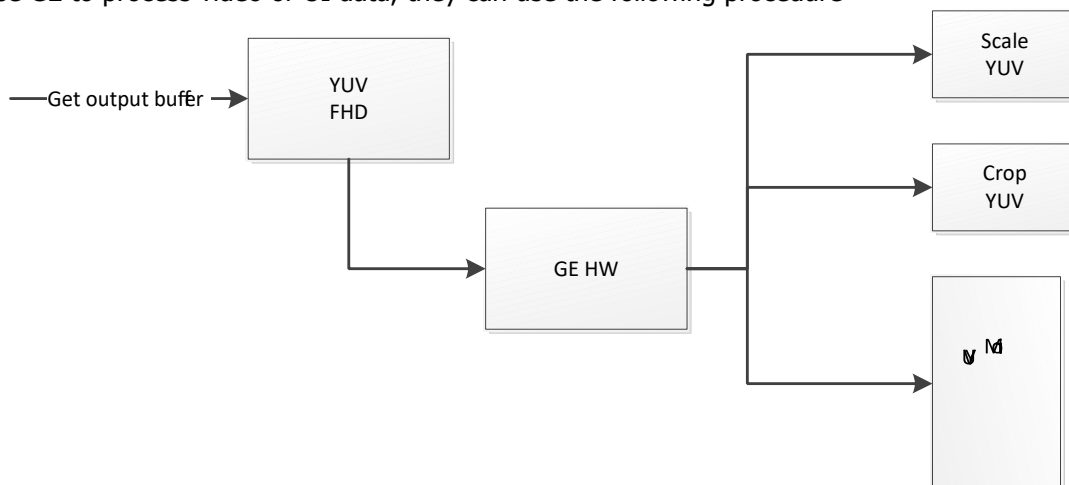


The other operations are generally consistent with standard 2D acceleration.

1.2. Flow diagram of invocation



If you want to use GE to process video or UI data, they can use the following procedure



1.3. Keyword Description

- Fence
The state in which a GFX operation needs to wait blocks the wait, but there is also an internal timeout mechanism to prevent latencies.
- Colorkey
Key colors, such as removing a color from the bitmap when moving, can be set to key colors.
- Pitch
The number of bytes per row of pixels, $(\text{pixel bits} / 8) * \text{Width}$. It also corresponds to the commonly used Stride and LineLength.

2. API LIST

The MI GFX module provides the following APIs:

API Name	Function
MI_GFX_Open	Open GFX device
MI_GFX_Close	Close GFX device
MI_GFX_WaitAllDone	Wait for one/all GFX tasks to complete
MI_GFX_QuickFill	Add quick fill operation to the task
MI_GFX_Bitblit	Add a raster bitmap to the task to perform move operations with additional functions
MI_GFX_SetPalette	Set palette for Index Color format
MI_GFX_CreateDev	Initialize GFX device
MI_GFX_DestroyDev	De-initialize GFX device
MI_GFX_GetARGB8888To1555AlphaThreshold	Get alpha threshold value of ARGB8888 convert to ARGB1555
MI_GFX_SetARGB8888To1555AlphaThreshold	Set alpha threshold value of ARGB8888 convert to ARGB1555
MI_GFX_GetARGB1555To8888AlphaValue	Get alpha value of ARGB1555 convert to ARGB8888
MI_GFX_SetARGB1555To8888AlphaValue	Set alpha value of ARGB1555 convert to ARGB8888
MI_GFX_GetInitialScalingCoeff	Get initial scaling coefficient
MI_GFX_SetInitialScalingCoeff	Set initial scaling coefficient
MI_GFX_DrawLine	Add line drawing operations to the task

2.1. MI_GFX_Open

➤ Function

Call this interface to open GFX device and initialize hardware system

➤ Syntax

```
MI_S32 MI_GFX_Open(MI\_GFX\_DEV GfxDevId);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

- Before performing any GFX related operations, call this interface and make sure GFX device is opened.
- Calling this interface after the GFX device has been initialized will return a "module initialized" message.

➤ Example

```
1. MI_S32 s32Ret = 0;
2. #define DEFAULT_GFX_DEV_ID (0)
3. #define DEFAULT_SOC_ID (0)
4.
5. MI_SYS_Init(DEFAULT_SOC_ID);
6. /* open GFX device*/
7. s32Ret = MI_GFX_Open(DEFAULT_GFX_DEV_ID);
8. if (MI_SUCCESS != s32Ret)
9. {
10.     return -1;
11. }
12. /* close GFX device*/
13. MI_GFX_Close(DEFAULT_GFX_DEV_ID);
14. MI_SYS_Exit(DEFAULT_SOC_ID);
```

2.2. MI_GFX_Close

➤ Function

Call this interface to close GFX device

➤ Syntax

```
MI_S32 MI_GFX_Close(MI\_GFX\_DEV GfxDevId);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

Times of calling [MI_GFX_Open](#) and [MI_GFX_Close](#) must be consistent. Repeated calling is allowed. When the times of Open is Same as Close, the GFX API will no longer be able to be called.

➤ Example

N/A.

➤ Related API

N/A.

2.3. MI_GFX_WaitAllDone

➤ Function

Call this interface to wait for completion of all or specified GFX tasks.

➤ Syntax

```
MI_S32 MI_GFX_WaitAllDone(MI\_GFX\_DEV GfxDevId, MI_BOOL bWaitAllDone, MI_penU16 u16TargetFence);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
bWaitAllDone	Wait until all GFX tasks are completed	Input
u16TargetFence	Wait until specified fence is reached. This parameter takes effect only when bWaitAllDone is FALSE.	Input

➤ Return Value

- Zero: Successful

- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

This interface is a blocking interface which will block operations until all the GFX tasks are completed.

➤ Example

N/A.

2.4. MI_GFX_QuickFill

➤ Function

Fill in the color of u32ColorVal to the target area pstDstRect in the target bitmap pstDst to realize the function of color filling.

➤ Syntax

```
MI_S32 MI_GFX_QuickFill(MI_GFX_DEV GfxDevId, MI_GFX_Surface_t *pstDst,
MI_GFX_Rect_t *pstDstRect, MI_U32 u32ColorVal, MI_U16 *pu16Fence);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pstDst	Destination bitmap	Input
pstDstRect	Operating area in destination bitmap	Input
u32ColorVal	Color fill value	Input
pu16Fence	Returned fence pointer	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

- Before calling this interface, make sure to call [MI_GFX_Open](#) to open the GFX device.
- The target bitmap operation area cannot exceed the size of the target bitmap.
- If the pixel format of the target bitmap is RGB or ARGB, u32ColorVal from high to low is A8: R8: G8: B8. If the pixel format of the target bitmap is YUYV422, A8 is invalid, and R

corresponds to Y, G Corresponds to U, B corresponds to V.

➤ Example

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3. FILE *fp = NULL;
4. MI_PHY phyAddr;
5. void *pVirAddr = NULL;
6. MI_GFX_Surface_t stDst;
7. MI_GFX_Rect_t stDstRect;
8. MI_U32 u32ColorVal = 0xffff0000;
9. MI_U16 u16TargetFence;
10.
11. fp = fopen(SRC_FILE_NAME, "wb");
12.
13. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
14. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
15. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
16. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
17.
18. ExecFunc(MI_GFX_Open(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
19.
20. //fillrect
21. memset(&stDst, 0x0, sizeof(stDst));
22. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
23. stDst.u32Width = SRC_WIDTH;
24. stDst.u32Height = SRC_HEIGHT;
25. stDst.u32Stride = SRC_WIDTH * 2;
26. stDst.phyAddr = phyAddr;
27.
28. memset(&stDstRect, 0x0, sizeof(stDstRect));
29. stDstRect.s32Xpos = 100;
30. stDstRect.s32Ypos = 100;
31. stDstRect.u32Width = 100;
32. stDstRect.u32Height = 100;
33. ExecFunc(MI_GFX_QuickFill(DEFAULT_GFX_DEV_ID, &stDst, &stDstRect, u32ColorVal, &u16TargetFence), MI_SUCCESS);
34. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_GFX_DEV_ID, FALSE, u16TargetFence), MI_SUCCESS);
35. if (NULL != fp)
36. {
37.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
38.     fclose(fp);
39.     fp = NULL;
40. }
41. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);
42. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
43.
44. ExecFunc(MI_GFX_Close(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
45. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);

```

2.5. MI_GFX_GetARGB8888To1555AlphaThreshold

➤ Function

Get alpha threshold value.

When the source bitmap and the target bitmap do bitblit operations, regardless of the pixel

format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap with the pixel format of ARGB8888, and then output the result of the intermediate bitmap operation to the target bitmap. When the pixel format of the target image is ARGB1555, if the alpha value of a pixel after the intermediate bitmap operation is less than this threshold, the alpha bit of the pixel output to the target bitmap is 0; if it is greater than or equal to this threshold, the alpha bit of the pixel is taken 1.

➤ Syntax

```
MI_S32 MI_GFX_GetARGB8888To1555AlphaThreshold((MI_GFX_DEV GfxDevId, MI_U8
*pu8ThresholdValue);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pu8ThresholdValue	Pointer to alpha threshold value	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

N/A.

➤ Example

N/A.

2.6. MI_GFX_SetARGB8888To1555AlphaThreshold

➤ Function

Set alpha threshold value.

When the source bitmap and the target bitmap do bitblit operations, regardless of the pixel format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap with the pixel format of ARGB8888, and then output the result of the intermediate bitmap operation to the target bitmap. When the pixel format of the target image is ARGB1555, if the alpha value of a pixel after the intermediate bitmap operation is less than this threshold, the alpha bit of the pixel output to the target bitmap is 0; if it is greater than or equal to this threshold, the alpha bit of the pixel is taken 1.

➤ Syntax

```
MI_S32 MI_GFX_SetARGB8888To1555AlphaThreshold(MI_GFX_DEV GfxDevId, MI_U8
```

u8ThresholdValue);

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
u8ThresholdValue	Alpha threshold value	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

N/A.

➤ Example

N/A.

2.7. MI_GFX_BitBlit

➤ Function

Operate the specified area (pstSrcRect, pstDstRect) of the source bitmap (pstSrc) and the target bitmap (pstDst), and copy the calculated bitmap to the specified area (pstDstRect) of the target bitmap (pstDst).

➤ Syntax

```
MI_S32 MI_GFX_BitBlit(MI_GFX_DEV GfxDevId, MI_GFX_Surface_t *pstSrc, MI_GFX_Rect_t
*pstSrcRect, MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t *pstDstRect, MI_GFX_Opt_t
*pstOpt, MI_U16 *pu16Fence);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pstSrc	Source bitmap	Input
pstSrcRect	Source bitmap operation region	Input
pstDst	Destination bitmap	Input
pstDstRect	Destination bitmap operation region	Input
pstOpt	Operating parameter setting structure	Input
pu16Fence	Pointer to the returned fence	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: `mi_gfx.h` `mi_gfx_datatype.h`
- Library: `libmi_gfx.a/libmi_gfx.so`

➤ Note

- Before calling this interface, make sure to call [MI_GFX_Open](#) to open the GFX device.
- When the size of the source bitmap operation area is inconsistent with the target bitmap operation area, the source area is scaled according to the target area ratio, and then the operation is performed with the target area.
- If the clip operation is an in-area clip, the cut area must have a common intersection with the operation area, otherwise an error will be returned.
- The [MI_GFX_Opt_t](#) structure stores the configuration information of the GFX calculation function, such as: whether to use the colorkey and the configuration value of the colorkey; whether to perform clip operation and specify the clip area; whether to mirror or perform alpha blending, etc. The above operations can be enabled at same time.

➤ Example

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3.
4. FILE *fp = NULL;
5. FILE *dstfp = NULL;
6. MI_PHY phyAddr, phyAddr2;
7. void *pVirAddr = NULL, *pVirAddr2 = NULL;
8. MI_GFX_Surface_t stSrc, stDst;
9. MI_GFX_Rect_t stSrcRect, stDstRect;
10. MI_U16 u16TargetFence;
11.
12. fp = fopen(SRC_FILE_NAME, "wb");
13. dstfp = fopen(DST_FILE_NAME, "wb");
14.
15. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
16. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
17. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", DST_WIDTH*360*2, &phyAddr2), MI_SUCCESS);
18.
19. //MI_U32 phySrcAddr, phyDstAddr;
20. ExecFunc(MI_GFX_Open(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
21. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
22. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
23.
24. ExecFunc(MI_SYS_Mmap(phyAddr2, DST_WIDTH*DST_HEIGHT*2, &pVirAddr2, FALSE), MI_SUCCESS);
25. memset(pVirAddr2, 0x0F, DST_WIDTH*DST_HEIGHT*2);
26.
27. //bitblit
28. memset(&stSrc, 0x0, sizeof(stSrc));
29. stSrc.eColorFmt = E_MI_GFX_FMT_ARGB1555;

```

```

30. stSrc.u32Width = SRC_WIDTH;
31. stSrc.u32Height = SRC_HEIGHT;
32. stSrc.u32Stride = SRC_WIDTH * 2;
33. stSrc.phyAddr = phyAddr;
34.
35. memset(&stSrcRect, 0x0, sizeof(stSrcRect));
36. stSrcRect.s32Xpos = 100;
37. stSrcRect.s32Ypos = 100;
38. stSrcRect.u32Width = 300;
39. stSrcRect.u32Height = 300;
40.
41. memset(&stDst, 0x0, sizeof(stDst));
42. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
43. stDst.u32Width = DST_WIDTH;
44. stDst.u32Height = DST_HEIGHT;
45. stDst.u32Stride = DST_WIDTH * 2;
46. stDst.phyAddr = phyAddr2;
47.
48. memset(&stDstRect, 0x0, sizeof(stDstRect));
49. stDstRect.s32Xpos = 200;
50. stDstRect.s32Ypos = 100;
51. stDstRect.u32Width = 200;
52. stDstRect.u32Height = 100;
53.
54. ExecFunc(MI_GFX_BitBlit(DEFAULT_GFX_DEV_ID, &stSrc, &stSrcRect, &stDst, &stDstRect, NULL, &u16TargetFence), MI_SUCCESS);
55. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_GFX_DEV_ID, FALSE, u16TargetFence), MI_SUCCESS);
56. if (NULL != dstfp)
57. {
58.     fwrite(pVirAddr2, 1, DST_WIDTH*DST_HEIGHT*2, dstfp);
59.     fclose(dstfp);
60.     dstfp = NULL;
61. }
62.
63. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);
64. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
65.
66. ExecFunc(MI_SYS_Munmap(pVirAddr2, DST_WIDTH*DST_HEIGHT*2), MI_SUCCESS);
67. ExecFunc(MI_SYS_MMA_Free(phyAddr2), MI_SUCCESS);
68.
69. ExecFunc(MI_GFX_Close(DEFAULT_GFX_DEV_ID), MI_SUCCESS);
70. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);

```

2.8. MI_GFX_SetPalette

➤ Function

Set Palette for Index Color (I1/I2/I4/I8).

➤ Syntax

MI_S32 MI_GFX_SetPalette([MI_GFX_DEV](#) GfxDevId, [MI_GFX_ColorFmt_e](#) eColorFmt, [MI_GFX_Palette_t](#) *pstPalette);

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input

eColorFmt	Specific Index Color format: MI_GFX_ColorFmt_e	Input
pstPalette	Data array of corresponding Index Color	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

- Before calling this interface, make sure to call [MI_GFX_Open](#) to open the GFX device.
- pstPalette is an array of [MI_GFX_PaletteEntry_t](#) with a capacity of 256. Each [MI_GFX_PaletteEntry_t](#) represents a color, and the corresponding subscript in the array represents the index of the color. However, I1/I2/I4 and I8 correspond to one Palette respectively. The former can support a maximum of 16 colors, while the latter can support a maximum of 256 colors. The eColorFmt parameter can be used to distinguish which Palette is set.

➤ Example

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. MI_GFX_Palette_t myPal;
3. MI_GFX_PaletteEntry_t index_0;
4. MI_GFX_PaletteEntry_t index_1;
5. index_0.RGB.u8A = 0x40;
6. index_0.RGB.u8R = 0xFF;
7. index_0.RGB.u8G = 0;
8. index_0.RGB.u8B = 0;
9. index_1.RGB.u8A = 0x40;
10. index_1.RGB.u8R = 0;
11. index_1.RGB.u8G = 0xFF;
12. index_1.RGB.u8B = 0;
13.
14. myPal.aunPalette[0] = index_0;
15. myPal.aunPalette[1] = index_1;
16. myPal.u16PalStart = 0
17. myPal.u16PalEnd = 1;
18. MI_GFX_SetPalette(DEFAULT_GFX_DEV_ID, E_MI_GFX_FMT_I8, &myPal);

```

2.9. MI_GFX_CreateDev

➤ Function

Initialize GFX device.

➤ Syntax

MI_S32 MI_GFX_CreateDev([MI_GFX_DEV](#) GfxDevId, [MI_GFX_DevAttr_t](#) *pstDevAtt);

➤ Parameters

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pstDevAttr	Initialization device parameter	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

N/A

➤ Example

N/A

➤ Related API

N/A

2.10. MI_GFX_DestroyDev

➤ Function

De-initialize GFX device.

➤ Syntax

```
MI_S32 MI_GFX_DestroyDev(MI\_GFX\_DEV GfxDevId);
```

➤ Parameters

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

- This interface should be called after the device has been initialized; otherwise, a failed message will be returned.
- If this interface is not called after the app exited, the GFX device will be auto de-initialized.

➤ Example

N/A

➤ Related API

N/A

2.11. MI_GFX_GetARGB1555To8888AlphaValue

➤ Function

Get the alpha conversion value.

When the source bitmap and the target bitmap do bitblit operations, regardless of the pixel format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap with the pixel format of ARGB8888, and then output the result of the intermediate bitmap operation to the target bitmap. When the source bitmap or target image pixel format is ARGB1555, if the alpha bit of ARGB1555 is 0, the alpha of ARGB8888 after conversion is 0; if the alpha bit of ARGB1555 is 1, the alpha of ARGB8888 after conversion is the alpha conversion value.

➤ Syntax

```
MI_S32 MI_GFX_GetARGB1555To8888AlphaValue(MI\_GFX\_DEV GfxDevId, MI_U8
*pu8AlphaValue);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pu8AlphaValue	Pointer to alpha value	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

Not supported currently

➤ Example

N/A.

➤ Related API

N/A.

2.12. MI_GFX_SetARGB1555To8888AlphaValue

➤ Function

Set the alpha conversion value.

When the source bitmap and the target bitmap do bitblt operations, regardless of the pixel format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap with the pixel format of ARGB8888, and then output the result of the intermediate bitmap operation to the target bitmap. When the source bitmap or target image pixel format is ARGB1555, if the alpha bit of ARGB1555 is 0, the alpha of ARGB8888 after conversion is 0; if the alpha bit of ARGB1555 is 1, the alpha of ARGB8888 after conversion is the alpha conversion value.

➤ Syntax

```
MI_S32 MI_GFX_SetARGB1555To8888AlphaValue(MI_GFX_DEV GfxDevId, MI_U8
u8AlphaValue);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
u8AlphaValue	alpha value	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

Not supported currently

➤ Example

N/A.

➤ Related API

N/A.

2.13. MI_GFX_GetInitialScalingCoeff

➤ Function

Gets the scaling coefficient. When the bitblit operation requires scaling, the image geometric center offset of bilinear interpolation = scaling coefficient /1000.

➤ Syntax

```
MI_S32 MI_GFX_GetInitialScalingCoeff(MI\_GFX\_DEV GfxDevId, MI_U16
*pu16ScalingCoefficient);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input
pu16ScalingCoefficient	Pointer to scaling coefficient	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code](#) for details

➤ Requirement

- Header: mi_gfx.h mi_gfx_datatype.h
- Library: libmi_gfx.a/libmi_gfx.so

➤ Note

Not supported currently

➤ Example

N/A.

➤ Related API

N/A.

2.14. MI_GFX_SetInitialScalingCoeff

➤ Function

Set the scaling coefficient. When the bitblit operation requires scaling, the image geometric center offset of bilinear interpolation = scaling coefficient /1000. The default value is 500.

➤ Syntax

```
MI_S32 MI_GFX_SetInitialScalingCoeff(MI\_GFX\_DEV GfxDevId, MI_U16
u16ScalingCoefficient);
```

➤ Parameter

Parameter Name	Description	Input/Output
GfxDevId	GFX device id	Input

u16ScalingCoefficient	Scaling coefficient	Input
-----------------------	---------------------	-------

- Return Value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Requirement
 - Header: mi_gfx.h mi_gfx_datatype.h
 - Library: libmi_gfx.a/libmi_gfx.so
- Note

Not supported currently
- Example

N/A.
- Related API

N/A.

2.15. MI_GFX_DrawLine

- Function

Add line drawing operations to the task.
- Syntax


```
MI_S32 MI_GFX_DrawLine(MI\_GFX\_Surface\_t *pstDst, MI\_GFX\_Line\_t *pstLine, MI_U16 *pu16Fence);
```

- Parameter

Parameter Name	Description	Input/Output
pstDst	Target bitmap	Input
pstLine	Line attributes	Input
pu16Fence	Return pointer to Fence	Output

- Return Value
 - Zero: Successful
 - Non-zero: Failed, see [error code](#) for details
- Requirement
 - Header: mi_gfx.h
 - Library: libmi_gfx.a/libmi_gfx.so
- Note
 - Before calling this interface, make sure to call MI_GFX_Open to open the GFX device.

- The line drawing area cannot exceed the size of the target bitmap.
- The end point of the straight line will not be drawn.
- If the pixel format of the target bitmap is RGB or ARGB, u32ColorFrom/u32ColorTo from high to low is A8: R8: G8: B8. If the pixel format of the target bitmap is YUYV422, A8 is invalid, and R corresponds to Y, G corresponds to U, B corresponds to V.

➤ Example

```

1. #define DEFAULT_GFX_DEV_ID (0)
2. #define DEFAULT_SOC_ID (0)
3.
4. FILE *fp = NULL;
5. MI_GFX_Surface_t stDst;
6. MI_GFX_Line_t stLine;
7. MI_U16 u16TargetFence;
8. MI_PHY phyAddr;
9. void *VirAddr;
10.
11. ExecFunc(MI_SYS_Init(DEFAULT_SOC_ID), MI_SUCCESS);
12. fp = fopen(SRC_FILE_NAME, "wb");
13. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0", SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
14.
15. //MI_U32 phySrcAddr, phyDstAddr;
16. ExecFunc(MI_GFX_Open(DEFAULT_SOC_ID), MI_SUCCESS);
17. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2, &pVirAddr, FALSE), MI_SUCCESS);
18. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
19.
20. //draw line
21. memset(&stDst, 0x0, sizeof(stDst));
22. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
23. stDst.u32Width = SRC_WIDTH;
24. stDst.u32Height = SRC_HEIGHT;
25. stDst.u32Stride = SRC_WIDTH * 2;
26. stDst.phyAddr = phyAddr;
27.
28. memset(&stLine, 0x0, sizeof(stLine));
29. stLine.stPointFrom.s16x = 0;
30. stLine.stPointFrom.s16y = 0;
31. stLine.stPointTo.s16x = 100;
32. stLine.stPointTo.s16y = 100;
33. stLine.u16Width = 1;
34. stLine.bColorGradient = TRUE;
35. stLine.bColorFrom = 0xffff0000;
36. stLine.bColorTo = 0xffff00ff;
37.
38. ExecFunc(MI_GFX_DrawLine(DEFAULT_SOC_ID, &stDst, &stLine, &u16TargetFence), MI_SUCCESS);
39. ExecFunc(MI_GFX_WaitAllDone(DEFAULT_SOC_ID, FALSE, u16TargetFence), MI_SUCCESS);
40. if (NULL != fp)
41. {
42.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
43.     fclose(fp);
44.     fp = NULL;
45. }
46. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2), MI_SUCCESS);
47. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
48.
49. ExecFunc(MI_GFX_Close(DEFAULT_SOC_ID), MI_SUCCESS);

```

```
50. ExecFunc(MI_SYS_Exit(DEFAULT_SOC_ID), MI_SUCCESS);
```

3. GFX DATA TYPE

The GFX related data types are defined in the table below:

Function	Description
MI_GFX_ColorFmt_e	Define pixel color format
MI_GFX_Surface_t	Define bitmap attribute structure
MI_GFX_Rect_t	Define the attribute structure of the operating area
MI_GFX_ColorKeyOp_e	Define Colorkey operation mode
MI_GFX_ColorKeyValue_t	Define the Colorkey color attribute structure
MI_GFX_ColorKeyInfo_t	Define the Colorkey attribute structure
MI_GFX_Rotate_e	Define the image rotation angle
MI_GFX_Mirror_e	Define image mirror mode
MI_GFX_DfbBldOp_e	Define Alpha Blending mode
MI_Gfx_DfbBlendFlags_e	Define Alpha Blending flag
MI_GFX_Opt_t	Define optional attribute structure for Bitblit operation
MI_GFX_PaletteEntry_t	Define palette color structure
MI_GFX_Palette_t	Define palette structure
MI_GFX_DevAttr_t	Define GFX device initialization parameters
MI_GFX_Point_t	Define point attribute structure
MI_GFX_Line_t	Define line attribute structure
MI_GFX_DEV	GFX device id

3.1. MI_GFX_ColorFmt_e

➤ Description

Define pixel color format.

➤ Definition

```
typedef enum
{
    E_MI_GFX_FMT_I1 = 0, /* MS_ColorFormat */
    E_MI_GFX_FMT_I2,
    E_MI_GFX_FMT_I4,
    E_MI_GFX_FMT_I8,
    E_MI_GFX_FMT_FABAFGBG2266,
    E_MI_GFX_FMT_1ABFGBG12355,
    E_MI_GFX_FMT_RGB565,
    E_MI_GFX_FMT_ARGB1555,
    E_MI_GFX_FMT_ARGB4444,
    E_MI_GFX_FMT_ARGB1555_DST,
    E_MI_GFX_FMT_ARGB8888,
    E_MI_GFX_FMT_RGBA5551,
    E_MI_GFX_FMT_RGBA4444,
    E_MI_GFX_FMT_ABGR8888,
    E_MI_GFX_FMT_BGRA5551,
    E_MI_GFX_FMT_ABGR1555,
    E_MI_GFX_FMT_ABGR4444,
    E_MI_GFX_FMT_BGRA4444,
    E_MI_GFX_FMT_BGR565,
    E_MI_GFX_FMT_RGBA8888,
    E_MI_GFX_FMT_BGRA8888,
    E_MI_GFX_FMT_YUV422_YUYV,
    E_MI_GFX_FMT_YUV422_YVYU,
    E_MI_GFX_FMT_YUV422_UYVY,
    E_MI_GFX_FMT_YUV422_VYUY,
    E_MI_GFX_FMT_MAX
} MI_GFX_ColorFmt_e;
```

➤ Note

Color format supported by corresponding surface, which, during bitblit operation, corresponds to SrcSurface, DstSurface color format. I1/I2/I4/I8 can not be the color format of DstSurface.

➤ Related Data Type and Interface

[MI_GFX_ColorKeyInfo_t](#) , [MI_GFX_Surface_t](#) , [MI_GFX_SetPalette](#)

3.2. MI_GFX_Surface_t

➤ Description

Define bitmap attribute structure.

➤ Definition

```
typedef struct MI_GFX_Surface_s
{
    MI_PHY phyAddr;
    MI\_GFX\_ColorFmt\_e eColorFmt;
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32Stride;
} MI_GFX_Surface_t;
```

➤ Member

Member	Description
u32PhyAddr	The bitmap corresponds to the physical address of the memory.
eColorFmt	Bitmap pixel color format.
u32Height	The height of the bitmap.
u32Width	The width of the bitmap.
u32Stride	The stride of the bitmap.

➤ Note

- When the pixel format size of the bitmap is greater than or equal to 1Byte, the physical address of the memory corresponding to the bitmap and Stride must be aligned according to the pixel format size; when the pixel format size of the bitmap is less than 1Byte, the physical address of the memory corresponding to the bitmap and Stride need to be aligned according to 1Byte.
- The horizontal starting position and width of the bitmap in YUYV422 format must be even numbers.

➤ Related Data Type and Interface

[MI_GFX_QuickFill](#) , [MI_GFX_DrawLine](#) , [MI_GFX_BitBlit](#)

3.3. [MI_GFX_Rect_t](#)

➤ Description

Define the attribute structure of the operating area.

➤ Definition

```
typedef struct MI_GFX_Rect_s
{
    MI_S32 s32Xpos;
    MI_S32 s32Ypos;
    MI_U32 u32Width;
    MI_U32 u32Height;
} MI_GFX_Rect_t;
```

➤ Member

Member	Description
--------	-------------

s32Xpos	The starting abscissa of the operation region, in pixels. Valid range: [0, bitmap width)
s32Ypos	The starting ordinate of the operation region, in pixels. Valid range: [0, bitmap height)
u32Width	Width of the operation region, in pixels.
u32Height	Height of the operation region, in pixels.

➤ Note

The operating range must be within the range of the bitmap.

➤ Related Data Type and Interface

[MI_GFX_Opt_t](#) , [MI_GFX_QuickFill](#) , [MI_GFX_BitBlit](#)

3.4. MI_GFX_ColorKeyOp_e

➤ Description

Define Colorkey operation mode.

➤ Definition

```
typedef enum
{
    E_MI_GFX_RGB_OP_EQUAL = 0,
    E_MI_GFX_RGB_OP_NOT_EQUAL,
    E_MI_GFX_ALPHA_OP_EQUAL,
    E_MI_GFX_ALPHA_OP_NOT_EQUAL,
    E_MI_GFX_ARGB_OP_EQUAL,
    E_MI_GFX_ARGB_OP_NOT_EQUAL,
    E_MI_GFX_CKEY_OP_BUTT,
} MI_GFX_ColorKey_e;
```

➤ Member

Member	Description
E_MI_GFX_RGB_OP_EQUAL	RGB equal colorkey operation
E_MI_GFX_RGB_OP_NOT_EQUAL	RGB unequal colorkey operation
E_MI_GFX_ALPHA_OP_EQUAL	ALPHA equal colorkey operation
E_MI_GFX_ALPHA_OP_NOT_EQUAL	ALPHA unequal colorkey operation
E_MI_GFX_ARGB_OP_EQUAL	ARGB equal colorkey operation, See the note
E_MI_GFX_ARGB_OP_NOT_EQUAL	ARGB unequal colorkey operation, See the note
E_MI_GFX_CKEY_OP_BUTT	Invalid colorkey operation

➤ Note

- When performing color key operation against bitmap, you can set the foreground and background, and the key value can be equal or not.

- The selection criteria for E_MI_GFX_ARGB_OP_EQUAL are as follows:
Ikayaki:
srcSurface((Apixel == Acolorkey) || (RGBpixel == RGBcolorkey))
DSTSurface(ARGBpixel == ARGBcolorkey)
Muffin/Mochi:
(ARGBPixel == ARGBColorKey)
- The selection criteria for E_MI_GFX_ARGB_OP_NOT_EQUAL are as follows:
Ikayaki:
srcSurface(ARGBpixel != ARGBcolorkey)
DSTSurface((Apixel != Acolorkey) || (RGBpixel != RGBcolorkey))
Muffin/Mochi:
(ARGBPixel != ARGBColorKey)

➤ Related Data Type and Interface

[MI_GFX_ColorKeyInfo_t](#)

3.5. MI_GFX_ColorKeyValue_t

➤ Description

Define the Colorkey color attribute structure.

➤ Definition

```
typedef struct MI_GFX_ColorKey_s
{
    MI_U32 u32ColorStart;
    MI_U32 u32ColorEnd;
} MI_GFX_ColorKeyValue_t;
```

➤ Member

Member	Description
u32ColorStart	ColorKey color range start value
u32ColorEnd	ColorKey color range end value

➤ Note

When only one single color value is to go through colorkey operation, set u32ColorStart = u32ColorEnd = colorVal.

➤ Related Data Type and Interface

[MI_GFX_ColorKeyInfo_t](#)

3.6. MI_GFX_ColorKeyInfo_t

➤ Description

Define the Colorkey attribute structure.

➤ Definition

```
typedef struct MI_GFX_ColorKeyInfo_s
{
    MI_BOOL bEnColorKey;
    MI\_GFX\_ColorKeyOp\_e eCKeyOp;
    MI\_GFX\_ColorFmt\_e eCKeyFmt;
    MI\_GFX\_ColorKeyValue\_t stCKeyVal;
} MI_GFX_ColorKeyInfo_t;
```

➤ Member

Member	Description
bEnColorKey	ColorKey operation enable
eCKeyOp	ColorKey operation mode
stCKeyVal	ColorKey color range
eCKeyFmt	ColorKey color format

➤ Note

N/A.

➤ Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.7. MI_GFX_Rotate_e

➤ Description

Define the image rotation angle.

➤ Definition

```
typedef enum
{
    E_MI_GFX_ROTATE_0 = 0,
    E_MI_GFX_ROTATE_90,
    E_MI_GFX_ROTATE_180,
    E_MI_GFX_ROTATE_270
} MI_GFX_Rotate_e;
```

➤ Member

N/A.

➤ Note

N/A.

- Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.8. MI_GFX_Mirror_e

- Description

Define image mirror mode.

- Definition

```
typedef enum
{
    E_MI_GFX_MIRROR_NONE = 0,
    E_MI_GFX_MIRROR_HORIZONTAL,
    E_MI_GFX_MIRROR_VERTICAL,
    E_MI_GFX_MIRROR_BOTH,
    E_MI_GFX_MIRROR_MAX
} MI_GFX_Mirror_e;
```

- Member

Member	Description
E_MI_GFX_MIRROR_NONE	Output image mirror operation disabled
E_MI_GFX_MIRROR_HORIZONTAL	Output image horizontal mirror
E_MI_GFX_MIRROR_VERTICAL	Output image vertical mirror
E_MI_GFX_MIRROR_BOTH	Output image horizontal + vertical mirror
E_MI_GFX_MIRROR_MAX	Invalid mirror type

- Note

N/A.

- Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.9. MI_GFX_DfbBldOp_e

- Description

Define Alpha Blending mode.

- Definition

```
typedef enum
{
    E_MI_GFX_DFB_BLD_ZERO = 0,
    E_MI_GFX_DFB_BLD_ONE,
    E_MI_GFX_DFB_BLD_SRCCOLOR,
```

```

E_MI_GFX_DFB_BLD_INVSRCOLOR,
E_MI_GFX_DFB_BLD_SRCALPHA,
E_MI_GFX_DFB_BLD_INVSRCALPHA,
E_MI_GFX_DFB_BLD_DESTALPHA,
E_MI_GFX_DFB_BLD_INVDESTALPHA,
E_MI_GFX_DFB_BLD_DESTCOLOR,
E_MI_GFX_DFB_BLD_INVDESTCOLOR,
E_MI_GFX_DFB_BLD_SRCALPHASAT,
E_MI_GFX_DFB_BLD_NONE,
E_MI_GFX_DFB_BLD_MAX,
} MI_GFX_DfbBldOp_e;

```

➤ Member

Member	Description
E_MI_GFX_DFB_BLD_OP_ZERO	argb *= 0.0 Same as directfb DSBF_ZERO.
E_MI_GFX_DFB_BLD_OP_ONE	argb *= 1.0 Same as directfb DSBF_ONE.
E_MI_GFX_DFB_BLD_OP_SRCOLOR	argb *= Sargb Same as directfb DSBF_SRCOLOR.
E_MI_GFX_DFB_BLD_OP_INVSRCOLOR	argb *= 1.0 - Sargb Same as directfb DSBF_INVSRCOLOR.
E_MI_GFX_DFB_BLD_OP_SRCALPHA	argb *= Saaaa Same as directfb DSBF_SRCALPHA.
E_MI_GFX_DFB_BLD_OP_INVSRCALPHA	argb *= 1.0 - Saaaa Same as directfb DSBF_INVSRCALPHA.
E_MI_GFX_DFB_BLD_OP_DESTCOLOR	argb *= Dargb Same as directfb DSBF_DESTCOLOR.
E_MI_GFX_DFB_BLD_OP_INVDESTCOLOR	argb *= 1.0 - Dargb Same as directfb DSBF_INVDESTCOLOR.
E_MI_GFX_DFB_BLD_OP_DESTALPHA	argb *= Daaaa Same as directfb DSBF_DESTALPHA.
E_MI_GFX_DFB_BLD_OP_INVDESTALPHA	argb *= 1.0 - Daaaa Same as directfb DSBF_INVDESTALPHA.
E_MI_GFX_DFB_BLD_OP_SRCALPHASAT	rgb *= min(Sa, 1-Da) Same as directfb DSBF_SRCALPHASAT.
E_MI_GFX_DFB_BLD_NONE	Invalid alpha blending mode
E_MI_GFX_BLD_MAX	argb *= 0.0 Same as directfb DSBF_ZERO.

➤ Note

When the foreground bitmap and background bitmap are superimposed, you can set the superimposition mode of Src channel and Dst channel respectively. Now supports 11 overlay

modes. Use [MI_GFX_DfbBldOp_e](#) to set the mode.

- Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.10. MI_Gfx_DfbBlendFlags_e

- Description

Define Alpha Blending flag.

- Definition

typedef enum

{

```

    E_MI_GFX_DFB_BLEND_NOFX = 0x00000000,
    E_MI_GFX_DFB_BLEND_COLORALPHA = 0x00000001,
    E_MI_GFX_DFB_BLEND_ALPHACHANNEL = 0x00000002,
    E_MI_GFX_DFB_BLEND_COLORIZE = 0x00000004,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY = 0x00000008,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTCOLOR = 0x00000010,
    E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY = 0x00000020,
    E_MI_GFX_DFB_BLEND_XOR = 0x00000040,
    E_MI_GFX_DFB_BLEND_DEMULTIPLY = 0x00000080,
    E_MI_GFX_DFB_BLEND_SRC_COLORKEY = 0x00000100,
    E_MI_GFX_DFB_BLEND_DST_COLORKEY = 0x00000200,
    E_MI_GFX_DFB_BLEND_MAX = 0x3FF

```

} MI_Gfx_DfbBlendFlags_e;

- Member

Member	Description
E_MI_GFX_DFB_BLEND_NOFX	No BLD operation
E_MI_GFX_DFB_BLEND_COLORALPHA	src alpha will operate $\text{src.a} = \text{const color.a}$ according to const color value; Same as directfb DSBLIT_BLEND_COLORALPHA.
E_MI_GFX_DFB_BLEND_ALPHACHANNEL	src alpha will operate $\text{src.a} = \text{src.a} * \text{const color.a}$ according to const color value; Same as directfb DSBLIT_BLEND_ALPHACHANNEL.
E_MI_GFX_DFB_BLEND_COLORIZE	Use const color as the global color, adjust the source color; Same as directfb DSBLIT_COLORIZE.
E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY	Use the source alpha to pre-multiply the source color $\text{src.r} = \text{src.r} * \text{src.a}$; Same as directfb DSBLIT_SRC_PREMULTIPLY.

Member	Description
E_MI_GFX_DFB_BLEND_SRC_PREMULTCOLOR	Use const alpha to premultiply the source color $\text{src.r} = \text{src.r} * \text{const.a};$ Same as directfb DSBLIT_SRC_PREMULTCOLOR.
E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY	Use destination alpha to premultiply destination color $\text{dst.r} = \text{dst.r} * \text{dst.a};$ Same as directfb DSBLIT_DST_PREMULTIPLY.
E_MI_GFX_DFB_BLEND_XOR	Adjust and premultiply alpha $\text{src.a} \wedge \text{dst.a};$ Same as directfb DSBLIT_XOR
E_MI_GFX_DFB_BLEND_DEMULTIPLY	For conversion from premultiplied to non-premultiplied, it is best not to use such an operation $\text{x.r} = ((\text{int})\text{x.r} << 8) / ((\text{int})\text{x.a} + 1);$ Same as directfb DSBLIT_DEMULTIPLY.
E_MI_GFX_DFB_BLEND_SRC_COLORKEY	If the color of src is equal to const color, it will be keyed off
E_MI_GFX_DFB_BLEND_DST_COLORKEY	If the color of dst is equal to const color, it will be keyed off
E_MI_GFX_DFB_BLEND_MAX	All the above actions will be enabled

- Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.11. MI_GFX_BitBltnode_e

- Description

Define Bitblt interpolation mode.

- Definition

```
typedef enum
{
    E_MI_GFX_BITBLT_MODE_BILINEAR,
    E_MI_GFX_BITBLT_MODE_NEAREST,
} MI_GFX_BitBltnode_e;
```

- Member

Member	Description
E_MI_GFX_BITBLT_MODE_BILINEAR	Bilinear interpolation mode
E_MI_GFX_BITBLT_MODE_NEAREST	Nearest interpolation mode

- Related Data Type and Interface

[MI_GFX_Opt_t](#)

3.12. MI_GFX_Opt_t

➤ Description

Define optional attribute structure for Bitblt operation.

➤ Definition

```
typedef struct MI_GFX_Opt_s
{
    MI_GFX_Rect_t stClipRect;
    MI_GFX_ColorKeyInfo_t stSrcColorKeyInfo;
    MI_GFX_ColorKeyInfo_t stDstColorKeyInfo;
    MI_GFX_DfbBldOp_e eSrcDfbBldOp;
    MI_GFX_DfbBldOp_e eDstDfbBldOp;
    MI_GFX_Mirror_e eMirror;
    MI_GFX_Rotate_e eRotate;
    MI_Gfx_DfbBlendFlags_e eDFBBlendFlag;
    MI_GFX_BitBltMode_e eMode;
    MI_U32 u32GlobalSrcConstColor;
    MI_U32 u32GlobalDstConstColor;
} MI_GFX_Opt_t;
```

➤ Member

Member	Description
eDFBBlendFlag	Dfb blending operation flags
stSrcColorKeyInfo	Source surface colorkey operation
stDstColorKeyInfo	Destination surface colorkey operation
stClipRect	Clip region definition
eMirror	Image mirror type
eSrcDfbBldOp	Source surface BLEND operation type
eDstDfbBldOp	Destination surface BLEND operation type
eRotate	Image rotation angle: 0, 90, 180, 270°
eDFBBlendFlag	Blending related operations, such as XOR, AlphaBlending, Colorize, etc.
eMode	Bitblt interpolation mode.
u32GlobalSrcConstColor	const color for dfb blending operation
u32GlobalDstConstColor	const color for dfb blending operation

➤ Note

Currently only u32GlobalSrcConstColor is valid.

➤ Related Data Type and Interface

[MI_GFX_BitBlit](#)

3.13. MI_GFX_PaletteEntry_t

➤ Description

RGB color definition for an Index Color

➤ Definition

```
typedef union
{
    /// ARGB8888 byte order
    struct
    {
        MI_U8 u8A;
        MI_U8 u8R;
        MI_U8 u8G;
        MI_U8 u8B;
    } RGB;
    // u8Data[0] = u8A
    // u8Data[1] = u8R
    // u8Data[2] = u8G
    // u8Data[3] = u8B
    MI_U8 u8Data[4];
} MI_GFX_PaletteEntry_t;
```

➤ Member

Member	Description
RGB	Color component structure for an Index Color
u8Data	Byte order RGB data for an Index Color

➤ Note

Please pay attention to the remark in the definition, which will help you understand the color value's memory layout.

➤ Related Data Type and Interface

[MI_GFX_Palette_t](#)

3.14. MI_GFX_Palette_t

➤ Description

Define palette structure.

➤ Definition

```
typedef struct MI_GFX_Palette_s
{
```

```

MI\_GFX\_PaletteEntry\_t aunPalette[256];
MI_U16 u16PalStart;
MI_U16 u16PalEnd;
}MI_GFX_Palette_t;

```

➤ Member

Member	Description
aunPalette	Array of RGB colors
u16PalStart	Starting index
u16PalEnd	Ending index

➤ Note

N/A.

➤ Related Data Type and Interface

[MI_GFX_PaletteEntry_t](#).

3.15. [MI_GFX_DevAttr_t](#)

➤ Description

GFX device initialization parameter.

➤ Definition

```

typedef struct MI_GFX_DevAttr_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_GFX_DevAttr_t;

```

➤ Member

Member	Description
u32DevId	Device ID
u8Data	Data pointer buffer

➤ Note

N/A.

➤ Related Data Type and Interface

[MI_GFX_CreateDev](#)

3.16. [MI_GFX_Point_t](#)

➤ Description

Define point attribute structure.

➤ Definition

```
typedef struct MI_GFX_Point_s
{
    MI_S16 s16x;
    MI_S16 s16y;
} MI_GFX_Point_t;
```

➤ Member

Member	Description
s16x	X coordinate
S16y	Y coordinate

➤ Note

N/A

➤ Related Data Type and Interface

[MI_GFX_Line_t](#)

3.17. MI_GFX_Line_t

➤ Description

Define line attribute structure.

➤ Definition

```
typedef struct MI_GFX_Line_s
{
    MI\_GFX\_Point\_t stPointFrom;
    MI\_GFX\_Point\_t srPointTo;
    MI_U16 u16Width;
    MI_BOOL bColorGradient;
    MI_U32 u32ColorFrom;
    MI_U32 u32ColorTo;
} MI_GFX_Line_t;
```

➤ Member

Member	Description
stPointFrom	Starting point of the straight line
srPointTo	The end of the straight line
u16Width	Straight line width
bColorGradient	Whether the color gradient
u32ColorFrom	Gradient starting color

Member	Description
u32ColorTo	Gradient end color

➤ Note

N/A

➤ Related Data Type and Interface

[MI_GFX_DrawLine](#)

3.18. MI_GFX_Dev

➤ Description

GFX device id.

➤ Definition

```
typedef MI_S32 MI_GFX_DEV;
```

➤ Member

N/A

➤ Note

Except for Muffin supports 2 devices, all the others only support 1.

➤ Related Data Type and Interface

All API interfaces.

4. GFX ERROR CODE

The GFX API error codes are as listed in the table below:

Error Code	Macro Definition	Description
0xA00D2003	MI_ERR_GFX_INVALID_PARAM	Invalid parameter
0xA00D2002	MI_ERR_GFX_DEV_BUSY	GFX device busy
0xA00D2220	MI_ERR_GFX_NOT_INIT	Device not initialized
0xA00D2221	MI_ERR_GFX_DRV_NOT_SUPPORT	Unsupported operation
0xA00D2222	MI_ERR_GFX_DRV_FAIL_FORMAT	Unsupported format
0xA00D2223	MI_ERR_GFX_NON_ALIGN_ADDRESS	Address not aligned
0xA00D2224	MI_ERR_GFX_NON_ALIGN_PITCH	Pitch not aligned
0xA00D2225	MI_ERR_GFX_DRV_FAIL_OVERLAP	Overlap region found in Option
0xA00D2226	MI_ERR_GFX_DRV_FAIL_STRETCH	Stretch failed
0xA00D2228	MI_ERR_GFX_DRV_FAIL_LOCKED	Device locked by other use
0xA00D2229	MI_ERR_GFX_DRV_FAIL_BLTADDR	Bitblit start address error

5. PROCFS INTRODUCTION

5.1. cat

➤ Debug info

cat /proc/mi_modules/mi_gfx/mi_gfx0

```
-----Gfx Dev Attr-----
module init count:  2
Device id:  0
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000

Device id:  1
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000
-----
```

➤ Debug info analysis

Record GFX current usage status and device attribute which can be dynamically got to debug and test.

➤ Parameter Description

Parameter		Description
device info	module init count	Device opened times. Since GFX will be called by multiple modules, and it is not certain which module will call GFX initialization first, the device can be opened multiple times. The opened times will be recorded to determine whether the device is required to be closed.
	Device id	GFX device ID
	ARGB1555 alpha threshold	Convert ARGB8888 to ARGB1555. If the alpha threshold in GFX is greater than N, then alpha is 1, otherwise, alpha is 0.
	ARGB1555 To ARGB8888 alpha value	Convert ARGB1555 to ARGB8888, GFX converts the pixel alpha whose alpha bit of ARGB1555 is 1 to ARGB8888 Alpha Value.
	Scaling coefficient	Image geometric center offset